

CAPSTONE PROJECT REPORT

LyricistGPT

A Specialized Conversational LLM for Hindi Film Lyrics

Combining Instruction Fine-Tuning with Retrieval-Augmented Generation

IndustryGPT: Specialized LLM Bot Using Pre-Trained Models

Domain: Media & Entertainment

Prepared by: Shourya Kumar

Date: 09-07-2026

Table of Contents

Contents

Abstract4

1. Introduction4

 1.1 Background and Motivation4

 1.2 Problem Statement4

 1.3 Objectives4

 1.4 Industry and Scope5

2. System Architecture Overview5

 2.1 End-to-End Pipeline5

3. Dataset Preparation.....5

 3.1 Corpus Design6

 3.2 Derived Fields6

 3.3 Data Quality and Encoding6

4. Instruction Dataset Construction6

5. Model Fine-Tuning.....7

 5.1 Base Model and Rationale7

 5.2 Parameter-Efficient Fine-Tuning (QLoRA)7

 5.3 Training Configuration7

 5.4 Training Results7

6. Embeddings and Vector Database.....8

 6.1 Multilingual Embeddings8

 6.2 FAISS Index.....8

 6.3 Retrieval Text.....8

7. Retrieval-Augmented Generation Pipeline.....8

8. Intent Router and Task-Specific Handlers8

 8.1 Motivation8

 8.2 Supported Intents.....9

 8.3 Routing Strategy9

 8.4 Hallucination Control.....9

9. Deployment9

10. Results and Demonstration10

 10.1 Lyricist Identification10

 10.2 Theme Analysis11

10.3 Song Recommendation (Retrieval-First).....	11
10.4 Original Lyric Generation	12
11. Challenges and Solutions	13
12. Limitations and Future Work	13
13. Conclusion.....	13

(Right-click → Update Field to populate page numbers after opening in Word.)

Abstract

This report presents **LyricistGPT**, an industry-specific conversational assistant built for the Hindi film lyrics domain within the Media & Entertainment sector. The system addresses a well-known limitation of general-purpose large language models (LLMs): when applied to narrow, culturally specific domains, they lack the specialised voice required for expert analysis and tend to *hallucinate* factual details such as song titles and lyricist attributions. LyricistGPT resolves both problems through a hybrid architecture that couples **parameter-efficient instruction fine-tuning** with **Retrieval-Augmented Generation (RAG)**.

A base model, Qwen2.5-1.5B-Instruct, was fine-tuned using the QLoRA method on a purpose-built instruction dataset of over 1,500 conversational examples derived from a curated corpus of approximately 350 songs spanning seven lyricists. A dense retrieval layer was constructed using BAAI/bge-m3 multilingual embeddings indexed in a FAISS vector store. At inference time, an **intent router** classifies each query into one of six task types and dispatches it to a dedicated, retrieval-grounded prompt handler. Recommendation and identification tasks are made retrieval-first to eliminate fabricated outputs. The complete system is deployed as an interactive Streamlit application. Training exhibited a monotonic decrease in both training and validation loss across five epochs, indicating effective learning without overfitting, and the deployed assistant demonstrably identifies lyricists, analyses themes and style, recommends songs, and generates original lyrics while grounding factual claims in retrieved evidence.

1. Introduction

1.1 Background and Motivation

Large Language Models have transformed conversational AI, yet their competence is broad rather than deep. In a specialised domain such as Hindi film lyricism—where meaning depends on literary devices, Hindi–Urdu register, and the distinctive signatures of individual poets—a general model produces two characteristic failures. First, it lacks the *analytical voice* of a domain expert. Second, and more seriously, it fabricates specifics: it will confidently attribute lyrics to the wrong writer or invent songs that do not exist. For an assistant whose value rests on trustworthiness, these are disqualifying flaws.

LyricistGPT was conceived to demonstrate that a modest, resource-efficient model can be elevated into a reliable domain specialist by combining two complementary techniques rather than relying on scale alone.

1.2 Problem Statement

Design and implement an industry-specific conversational bot that can (i) identify the likely lyricist of a passage of lyrics, (ii) explain a song’s theme and emotional content, (iii) analyse writing style and literary devices, (iv) recommend similar songs, and (v) generate original lyrics in a requested style—**while minimising factual hallucination** and remaining deployable on constrained hardware (a single Google Colab T4 GPU).

1.3 Objectives

1. Curate a clean, richly annotated, domain-specific dataset of Hindi film songs.

2. Convert the corpus into an instruction dataset and fine-tune a pre-trained LLM using a parameter-efficient method.
3. Build a multilingual dense-retrieval layer over the corpus to ground factual responses.
4. Introduce an intent router with task-specific prompts, including retrieval-first strategies for hallucination-prone tasks.
5. Deploy the assistant as an interactive application supporting live demonstration.

1.4 Industry and Scope

The chosen industry is **Media & Entertainment**, instantiated through the Hindi film music vertical. The scope is deliberately bounded to a curated corpus of seven lyricists, which makes the system an authoritative expert on that collection rather than a comprehensive encyclopaedia of Indian cinema—an intentional trade-off that maximises factual reliability within a well-defined boundary.

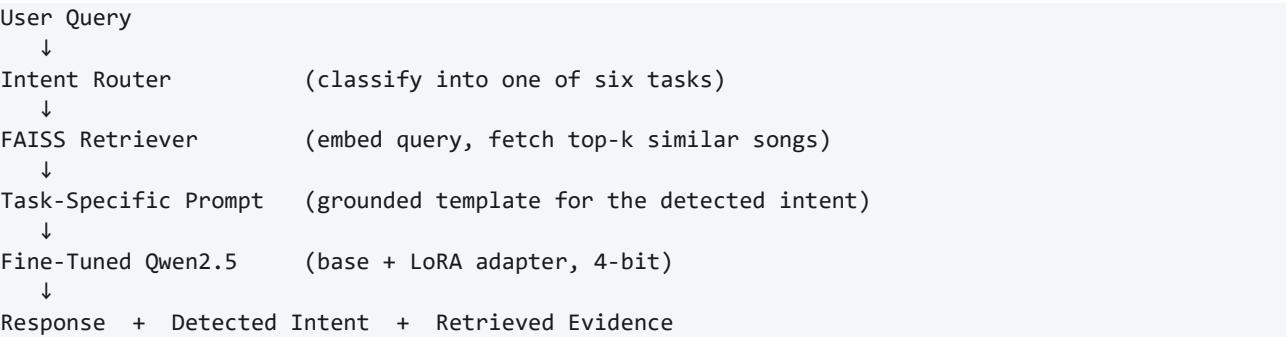
2. System Architecture Overview

LyricistGPT is a **hybrid** system. The design principle is a division of labour: fine-tuning supplies the expert *voice*, while retrieval supplies the *facts*. Each technique compensates for the other’s weakness.

Design thesis. Fine-tuning teaches the model how to speak like a lyrics analyst but cannot make a 1.5B-parameter model a reliable fact store. Retrieval supplies verifiable facts from the curated corpus but does not confer analytical style. Grounding a fine-tuned model on retrieved context yields answers that are both fluent in the domain and factually anchored.

2.1 End-to-End Pipeline

At inference time, a user query flows through the following stages:



The subsections that follow describe each phase in the order it was built.

□ **Figure 2.1 — System Architecture Diagram** *Optional: insert your architecture diagram / workflow figure here. [Paste screenshot here]*

3. Dataset Preparation

3.1 Corpus Design

A curated dataset of approximately 350 songs across seven lyricists was assembled. Each record is richly annotated to support both analysis and retrieval, with the following fields: *ID, Lyricist, Song Name, Era, Film Genre, Mood, Theme, Language Register, Literary Devices, a Hindi lyric excerpt (Devanagari), an English gloss, Style Notes, and a Source Reference.*

3.2 Derived Fields

Two derived fields were engineered to serve the downstream components:

- **document** — a structured knowledge document combining all attributes of a song into a single passage, used as the unit of retrieval.
- **combined_style** — a concatenation of mood, theme, language register, literary devices, and style notes, providing a compact semantic signature for similarity search.

3.3 Data Quality and Encoding

Devanagari text is fragile in tabular pipelines. Editing the corpus in spreadsheet software introduced encoding corruption of Hindi characters. This was resolved by standardising on **UTF-8 with BOM (utf-8-sig)** for all reads and writes. Schema inconsistencies—for example, divergent column names for the lyric fields—were standardised to a single canonical schema to prevent silent retrieval failures downstream.

Data-quality note. Encoding and schema normalisation are unglamorous but decisive: an undetected Devanagari corruption would silently degrade every embedding computed from that text, poisoning retrieval quality across the entire system.

4. Instruction Dataset Construction

Supervised fine-tuning requires conversational, instruction–response data. The structured corpus was therefore transformed into **ChatML-formatted conversations**. For each song, multiple instruction types were programmatically generated, expanding the effective training corpus from ~350 songs to over **1,500 instruction–response examples**.

The generated instruction types mirror the target capabilities of the deployed assistant:

- Identify the lyricist of a passage.
- Explain the theme and emotional content of a song.
- Analyse writing style and literary devices.
- Compare the styles of two lyricists.
- Recommend songs by mood or theme.
- Generate original lyrics in a specified style.

Aligning the training tasks with the deployment tasks ensures the fine-tuned model is practised precisely in the behaviours the intent router will later invoke.

5. Model Fine-Tuning

5.1 Base Model and Rationale

The base model is **Qwen2.5-1.5B-Instruct**. At 1.5 billion parameters it is compact enough to fine-tune and serve on a single T4 GPU, while its instruction-tuned lineage provides a strong conversational foundation. The modest size is a deliberate choice: the project’s thesis is that grounding and specialisation—not raw scale—deliver domain reliability.

5.2 Parameter-Efficient Fine-Tuning (QLoRA)

Rather than updating all model weights, the **QLoRA** technique was used. The base model is loaded in **4-bit NF4 quantisation** to reduce memory footprint, and a small set of trainable **Low-Rank Adaptation (LoRA)** matrices is inserted into the attention and MLP projections. Only these adapters are trained; the quantised base remains frozen. This reduces the trainable-parameter count by orders of magnitude, fits comfortably on a T4, and yields a compact adapter artifact that is trivial to store and load.

5.3 Training Configuration

Setting	Value
Base model	Qwen/Qwen2.5-1.5B-Instruct
Method	QLoRA (4-bit NF4 base + LoRA adapters)
Hardware	Google Colab, single T4 GPU
Epochs	5 (within the 25-epoch project cap)
Data	~1,500 ChatML instruction examples
Split	Training / validation
Artifact	LoRA adapter + tokenizer saved to Google Drive

5.4 Training Results

The model was trained for five epochs. Both training loss and validation loss decreased **monotonically and in tandem**, with validation loss never diverging upward from training loss. This is the signature of successful learning *without overfitting*: the model continued to improve on unseen validation examples throughout training. The observed trajectory fell from approximately 0.96 to 0.16 for training loss and from approximately 0.90 to 0.33 for validation loss.

 **Figure 5.1 — Training and Validation Loss** *Insert the screenshot of your loss table / curve from the fine-tuning notebook. [Paste screenshot here]*

Interpreting the loss gap. The final gap between training (0.16) and validation (0.33) loss is normal and healthy at this scale. Overfitting would manifest as validation loss rising while training loss falls; here both fell together, confirming genuine generalisation.

6. Embeddings and Vector Database

6.1 Multilingual Embeddings

The document field of each song was encoded into a dense vector using the **BAAI/bge-m3** embedding model. This model was selected for its strong multilingual capability: it represents English metadata and Devanagari lyric excerpts within a single shared semantic space, which is essential because user queries may arrive in either language.

6.2 FAISS Index

Embeddings were L2-normalised and indexed in a **FAISS IndexFlatIP** store. Normalisation converts the inner-product search into *cosine similarity*, the standard measure of semantic closeness. Because the corpus contains only a few hundred documents, an exact (flat) index is used; there is no need for approximate search, and exact retrieval maximises match quality. The index and its metadata are persisted so the application loads a prebuilt memory rather than recomputing embeddings at every launch.

6.3 Retrieval Text

For each song, the text submitted to the embedder places identity first (song name and lyricist), followed by the stylistic signature and the full document. Front-loading identity sharpens retrieval for attribution-type queries, while the stylistic and document content carries the semantic weight for theme, style, and recommendation queries.

7. Retrieval-Augmented Generation Pipeline

The RAG pipeline unites retrieval and generation in a disciplined sequence:

1. Encode the user query into an embedding using the same BGE-M3 model used for the corpus.
2. Retrieve the top-k most similar song documents from FAISS, each with a cosine-similarity score.
3. Construct a grounded prompt that embeds the retrieved context and instructs the model to answer only from that context.
4. Generate the response with the fine-tuned model.
5. Return the answer together with the detected intent and the retrieved evidence.

Why grounding matters. Retrieval places the correct facts in the model's context, transforming a closed-book task into an open-book one. This substantially reduces—though, as discussed in Section 8, does not by itself eliminate—the model's tendency to fabricate.

8. Intent Router and Task-Specific Handlers

8.1 Motivation

A single universal prompt proved inadequate: identification, thematic analysis, and recommendation are genuinely different tasks requiring different context and instructions. An **intent router** was therefore introduced to classify each query and dispatch it to a dedicated handler.

8.2 Supported Intents

Intent	User Goal
identify_lyricist	Determine the likely author of a passage of lyrics
analyze_theme	Explain the theme, meaning, and emotion
analyze_style	Describe writing style, imagery, and literary devices
recommend_songs	Suggest similar or thematically related songs
generate_lyrics	Compose original lyrics in a requested style
general	Comparisons, definitions, and other queries

8.3 Routing Strategy

Routing is a two-stage cascade. A fast, deterministic **keyword/heuristic classifier** handles the common, unambiguous cases at zero computational cost. Only when the heuristics are inconclusive does the system fall back to an **LLM-based classifier**. A dedicated check also detects pasted Devanagari lyrics with no explicit instruction and routes them to identification. This keeps routing both cheap and robust.

8.4 Hallucination Control

The most important design decision concerns the tasks most prone to fabrication. For **recommendation**, the pipeline is made **retrieval-first**: FAISS selects the actual songs from the corpus, and the model is permitted only to *explain why the retrieved songs fit*—it is explicitly forbidden from inventing titles. For **identification**, the handler anchors the answer to the top retrieved lyricist and runs deterministically, so the stated attribution agrees with the retrieved evidence. For **generation**, retrieved songs serve only as a style reference under an explicit no-copying instruction, protecting against reproduction of copyrighted lyrics.

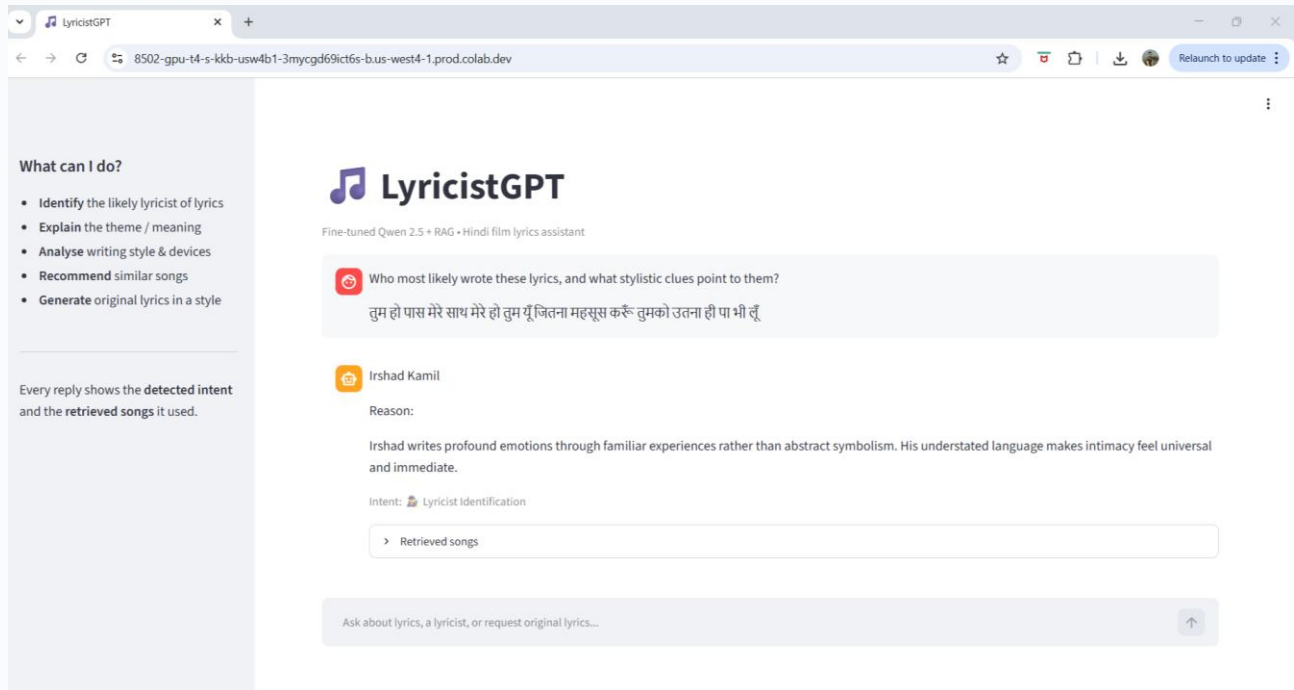
Key insight. Retrieval guarantees the model can see the correct answer; it does not guarantee the model will state it. Hallucination-prone tasks therefore require an additional constraint that removes the model’s freedom to invent—letting retrieval choose, and restricting the model to explanation.

9. Deployment

The complete system is deployed as an interactive **Streamlit** web application. The application loads the fine-tuned model, the embedding model, and the FAISS index once (cached for responsiveness) and reuses the exact router and handler logic developed during inference design. For every exchange it surfaces three elements: the generated answer, the **detected intent**, and the **retrieved songs with similarity scores**. Exposing the intent and evidence makes the system transparent, allowing a user to see the basis for each answer rather than trusting an opaque response.

The runtime architecture realises the pipeline of Section 2: *User* → *Streamlit UI* → *Intent Router* → *FAISS Retriever* → *Prompt Builder* → *Fine-Tuned Qwen* → *Response*.

□ Figure 9.1 — Streamlit Application Interface



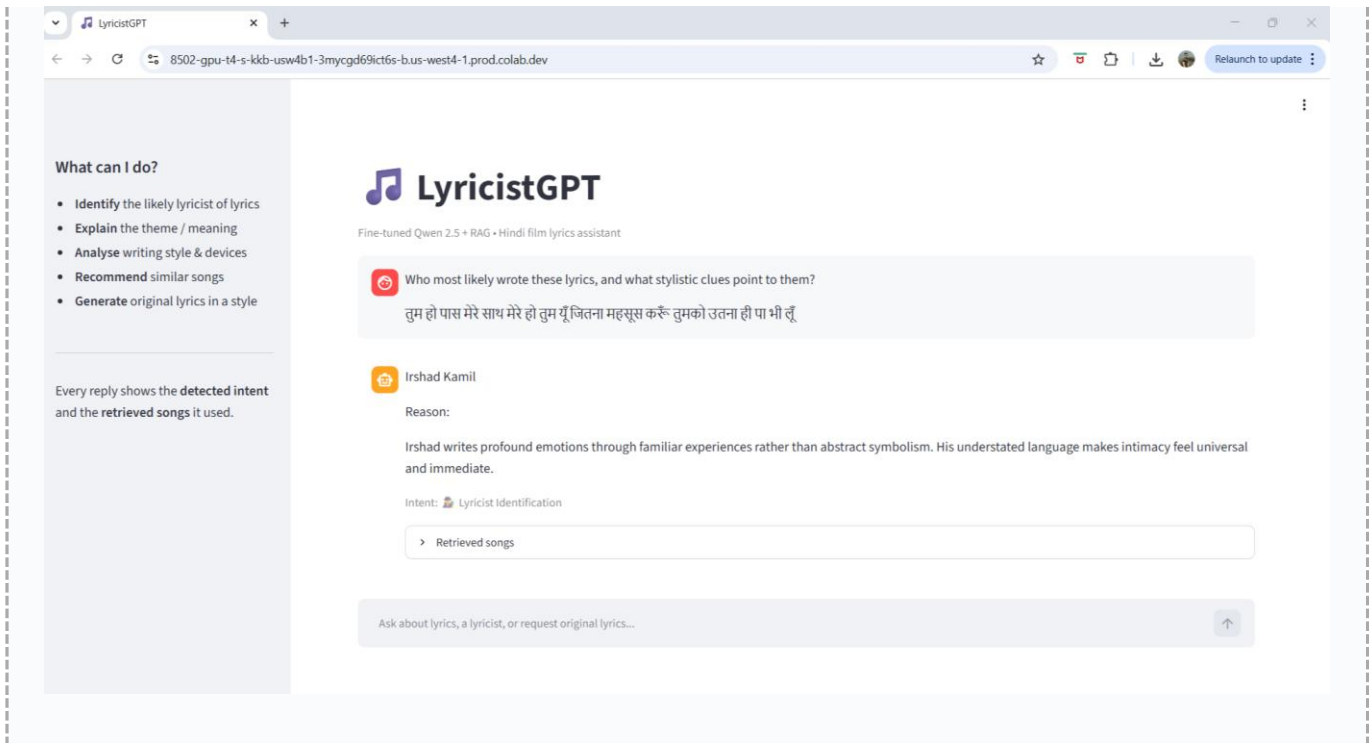
10. Results and Demonstration

This section presents representative interactions with the deployed assistant. Each example illustrates a different intent and the corresponding retrieval-grounded response. Paste your own chatbot screenshots into the labelled spaces below.

10.1 Lyricist Identification

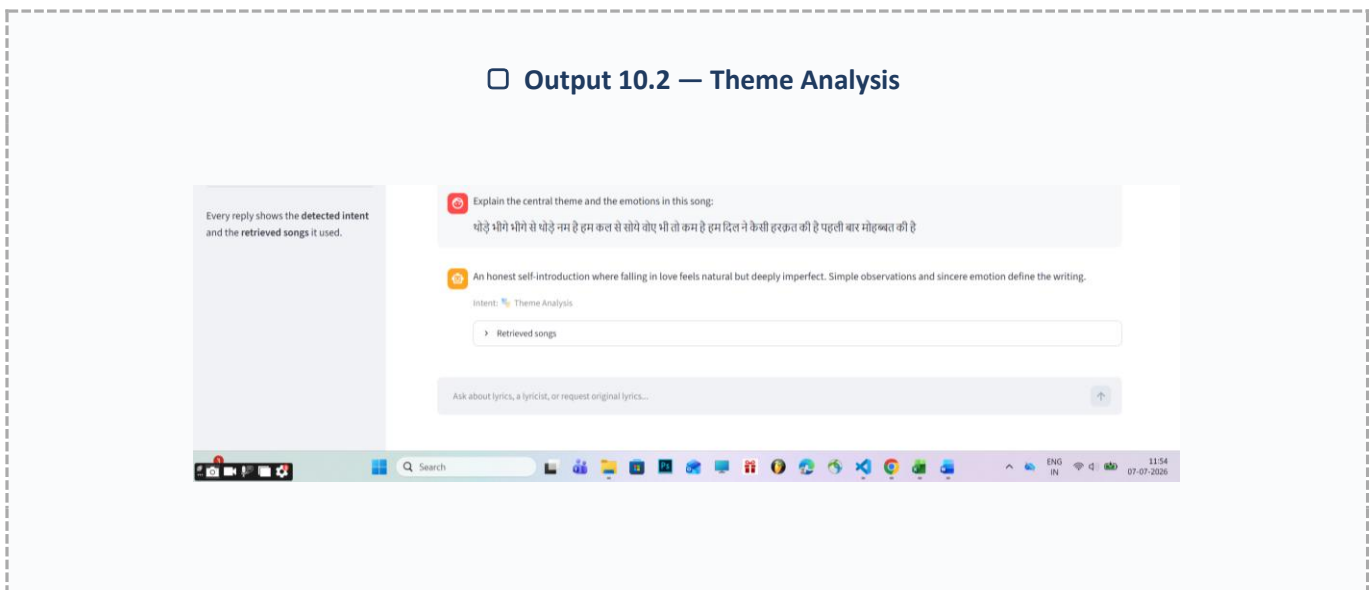
The user supplies a passage of lyrics; the router classifies the query as `identify_lyricist`, the retriever surfaces the most similar songs, and the response names the lyricist whose retrieved work best matches—justified by stylistic evidence and consistent with the retrieved table.

□ Output 10.1 — Identification



10.2 Theme Analysis

The assistant interprets the central theme and emotional content of a passage, grounded in retrieved context.



10.3 Song Recommendation (Retrieval-First)

Every recommended song is an item actually retrieved from the corpus; none are invented. This example is the clearest evidence of the hallucination-control design in Section 8.4.

□ Output 10.3 — Recommendation



Recommend songs about heartbreak and longing, and explain why each fits.



Songs should express vulnerability through quiet conversations rather than grand declarations.

Reasons:

Irshad's emotive writing keeps the language conversational while carrying immense emotional weight.

Intent: 🎧 Song Recommendation

▼ Retrieved songs

	songname	lyricist	score
0	Bairiya	Amitabh Bhattacharya	0.559
1	O Saathi Re	Gulzar	0.553
2	Beete Lamhein	Sayeed Quadri	0.552
3	Bekhayali	Irshad Kamil	0.549

10.4 Original Lyric Generation

The assistant composes original lines in a requested mood or style, using retrieved songs only as a stylistic reference and without reproducing existing lyrics.

□ Output 10.4 — Generation



write three original gulzar style lines with romantic mood in hindi



दिल बेकाबू क्यूँ... बादल पेश के भड़क जाये धाम में भूल जाये इस ख़तम को इशारत करे जिस प्यार को छिपाये भागे

Intent: ✍️ Lyric Generation

> Retrieved songs

Ask about lyrics, a lyricist, or request original lyrics...

11. Challenges and Solutions

Challenge	Resolution
Devanagari corruption in spreadsheets	Standardised on UTF-8 with BOM (utf-8-sig) for all I/O
Inconsistent column names for lyric fields	Enforced a single canonical schema across the pipeline
Quantisation / bf16 errors on Colab	Aligned library versions and pinned a working configuration
bitsandbytes dependency conflicts	Installed compatible versions; restarted runtime as required
Generic RAG prompt caused fabricated recommendations	Introduced intent routing with task-specific prompts
Recommendation hallucinations	Made recommendation retrieval-first; model only explains retrieved songs
Identification contradicting retrieved evidence	Anchored the handler to the top retrieved lyricist; deterministic decoding
Model persistence across sessions	Saved adapter and tokenizer to Google Drive

12. Limitations and Future Work

The assistant is grounded in a corpus of roughly 350 songs across seven lyricists; it is therefore an authoritative expert on that collection rather than the whole of Hindi cinema. As a 1.5B-parameter model, it depends on retrieval for factual reliability and should not be relied upon for free-recall facts outside the corpus.

Future work includes:

- Expanding the corpus to more lyricists, eras, and songs to broaden coverage.
- Adding a formal evaluation suite: lyricist-prediction accuracy and retrieval Top-1/Top-3 hit-rate, benchmarked against the base model and the fine-tuned-only variant.
- Extending the router with confidence thresholds and graceful clarification for ambiguous queries.
- Exploring adapter merging and further quantisation for lower-latency serving.

13. Conclusion

LyricistGPT demonstrates that a compact, resource-efficient language model can be transformed into a reliable domain specialist by combining instruction fine-tuning with retrieval-augmented generation. Fine-tuning supplies the expert analytical voice; retrieval supplies verifiable facts; the intent router ensures each task is handled by a purpose-built strategy; and retrieval-first design for hallucination-prone tasks keeps the assistant honest. The result is a transparent, interactive system—deployed via Streamlit—that identifies lyricists, analyses themes and style, recommends songs, and generates original lyrics, while grounding its factual claims in retrieved evidence. The project validates its central thesis: for narrow domains, grounding and specialisation deliver reliability that scale alone does not.